YOHPAL LIVE CREATOR STUDIO RELEASE 1.1A-RC2 Privacy Shield — People Image Blurring & Voice Anonymization

Daniel Macharia <danielmacharia43@gmail.com> Tue, 14 Jul at 11:33
To: <daniel@ics.ac.ke>

YOHPAL LIVE CREATOR STUDIO RELEASE 1.1A-RC2

Privacy Shield — People Image Blurring & Voice Anonymization

These two features should be added before launching the advanced video-editing suite. They can become a strong YohPal differentiator because they address privacy, child protection, journalism, safeguarding, entertainment, and responsible content creation.

The recommended product name is:

YohPal Privacy Shield

It should contain two independent but coordinated tools:

1. **Visual Identity Shield** — detects and obscures selected people throughout a video.
2. **Voice Identity Shield** — changes selected voices so the original speaker is difficult to recognize.

The user should be able to protect:

- All detected people
- All children or suspected minors
- One selected person
- Several selected people
- Everyone except the creator
- Faces appearing only during chosen time ranges
- One selected speaker
- All speakers except the creator
- Voices during selected timeline segments

---

1. Critical Product Distinction

“Voice blurring” should be presented technically as:

Voice anonymization

The feature should support two modes:

Privacy Mode

Designed for:

- Protected witnesses
- Children
- Survivors
- Whistle-blowers
- People without publication consent
- Sensitive interviews

Privacy Mode should prioritize identity protection over natural sound.

Creative Mode

Designed for:

- Comedy
- Characters
- Storytelling
- Entertainment
- Gaming
- Anonymous narration

Creative Mode may use playful presets but must not be presented as guaranteed anonymity.

---

2. Visual Identity Shield

Creator workflow

```
Import or record video
        ↓
Scan people
        ↓
Display detected person tracks
        ↓
Select people to protect
        ↓
Choose protection style
        ↓
Preview frame-by-frame
        ↓
Correct missed frames
        ↓
Render protection permanently
        ↓
Run privacy validation
        ↓
Publish
```

Protection styles

- Gaussian blur
- Pixelation
- Solid silhouette
- Emoji or avatar mask
- Mosaic
- Background replacement
- Full-body silhouette
- Custom sticker mask

For protected witnesses and minors, the recommended default is **strong blur or solid silhouette**, not a light beauty-style blur.

---

3. Person Tracking Requirements

Face detection alone is insufficient because people may:

- Turn away from the camera
- Cover their face
- Move rapidly
- Leave and re-enter the scene
- Appear in reflections
- Be visible only from the side
- Have their full body visible while the face is hidden

The engine should therefore combine:

```
Face detection
+ Face tracking
+ Person detection
+ Body tracking
+ Temporal identity association
+ Manual correction
```

Required behavior

- Detect every visible person.
- Assign a stable track ID.
- Track the person across frames.
- Allow the creator to tap a person once and protect them throughout the clip.
- Preserve protection when the person temporarily leaves the frame.
- Reconnect the same track when they return.
- Allow frame-by-frame mask correction.
- Flag low-confidence sections for creator review.

---

4. Minor Protection Mode

The system should not claim it can reliably determine a person's exact age from appearance.

Instead, provide:

```
Protect all people
Protect selected people
Protect possible minors
```

"Possible minor" should be treated as an AI warning requiring human confirmation.

Recommended interface:

"YohPal detected a person who may require child-protection review. Confirm whether their identity should be hidden."

The system should never automatically declare that someone is definitely a minor based solely on facial appearance.

---

5. Consent-Aware Workflow

Each detected person track should support:

```
Consent confirmed
Consent not confirmed
Consent withdrawn
Protection required
Not applicable
```

For professional, institutional, journalistic, or safeguarding accounts, publication may be blocked until every clearly identifiable person has one of:

- Valid consent
- Recorded legal/editorial justification
- Active Visual Identity Shield

This should be configurable by account type and jurisdiction.

---

6. Voice Identity Shield

Creator workflow

```
Analyse audio
     ↓
Separate speakers
     ↓
Display speaker tracks
     ↓
Select voice to protect
     ↓
Choose anonymization profile
     ↓
Preview
     ↓
Render transformed audio
     ↓
Run identity-leak check
     ↓
Publish
```

Required speaker capabilities

- Voice activity detection
- Speaker diarization
- Speaker separation where technically possible
- Manual timeline selection
- Selected-speaker transformation
- Background-noise preservation
- Subtitle synchronization after transformation
- Lip-sync preservation where practical

---

7. Voice Anonymization Presets

Privacy presets

Protected Witness — Maximum

- Strong formant transformation
- Pitch transformation
- Spectral shaping
- Temporal micro-variation
- Optional robotic carrier
- Original voice removed from the final mix

Protected Witness — Natural

- Moderate pitch shift
- Formant shift
- Timbre transformation
- Natural speech rhythm retained

Child Protection

- Strong anonymization
- No direct pitch-only transformation
- Formant and spectral modification
- Original audio track permanently excluded from published output

Anonymous Interview

- Natural but identity-obscured voice
- Speech intelligibility preserved

Creative presets

- Robot
- Deep voice
- Cartoon
- Alien
- Giant
- Tiny character
- Radio
- Mystery
- YohPal-generated custom character voice

---

8. Why Pitch Shifting Alone Is Not Enough

A simple pitch change may still preserve:

- Accent
- Speech rhythm
- Formants
- Pronunciation patterns
- Vocal mannerisms
- Background clues
- Original audio leakage

Privacy Mode should use a combination of:

```
Pitch modification
+ Formant transformation
+ Spectral-envelope modification
+ Timing micro-perturbation
+ Noise-floor management
+ Source-track removal
```

Even then, YohPal should state that anonymization reduces recognition risk but cannot guarantee absolute anonymity in every case.

---

9. High-Risk Privacy Checks

Before publishing protected content, the system should inspect for identity leaks beyond the face and voice.

Visual leak checks

- Reflections in mirrors or windows
- Visible ID cards
- Vehicle registration numbers
- School uniforms
- Tattoos
- House numbers
- Documents
- Computer screens
- Location signs
- Name badges

Audio leak checks

- Real names
- Addresses
- School names
- Employer names
- Phone numbers
- Background announcements
- Location clues
- Original voice leakage between transformed segments

Metadata checks

- GPS metadata
- Device metadata
- Original filename
- Creation location
- Embedded author information

For protected-witness mode, YohPal should strip unnecessary metadata from the final exported file.

---

10. Creator Studio Timeline Integration

The editor should add two new timeline track types:

```
Video clips
Face/person protection tracks
Audio tracks
Speaker protection tracks
Captions
Interactive objects
```

Example:

```
00:00 ──────────────────────────────── 00:30

Video:        [ Main interview clip            ]
Person P1:    [ Blur ████████████████████████ ]
Person P2:           [ Pixelate ███████        ]
Speaker S1:   [ Witness voice ████████████████ ]
Speaker S2:                      [ Normal voice ]
```

Every protection track must support:

- Start time
- End time
- Strength
- Style
- Track target
- Manual keyframes
- Confidence
- Review status

---

11. Domain Codebase

`privacy_protection_mode.dart`

```
enum PrivacyProtectionMode {
  privacy,
  creative,
}
```

visual_protection_track.dart

```
enum VisualProtectionStyle {
  gaussianBlur,
  pixelate,
  solidSilhouette,
  mosaic,
  emojiMask,
  avatarMask,
}

final class VisualProtectionKeyframe {
  const VisualProtectionKeyframe({
    required this.timestampMs,
    required this.left,
    required this.top,
    required this.width,
    required this.height,
    required this.confidence,
  });

  final int timestampMs;
  final double left;
  final double top;
  final double width;
  final double height;
  final double confidence;

  Map<String, dynamic> toJson() => {
        'timestampMs': timestampMs,
        'left': left,
        'top': top,
        'width': width,
        'height': height,
        'confidence': confidence,
      };
}

final class VisualProtectionTrack {
  const VisualProtectionTrack({
    required this.id,
    required this.personTrackId,
    required this.startMs,
    required this.endMs,
    required this.style,
    required this.strength,
    required this.keyframes,
    required this.reviewed,
    this.reason,
  });

  final String id;
  final String personTrackId;
  final int startMs;
  final int endMs;
  final VisualProtectionStyle style;

  /// Normalized between 0 and 1.
  final double strength;

  final List<VisualProtectionKeyframe> keyframes;
  final bool reviewed;
  final String? reason;

  bool appliesAt(int positionMs) {
    return positionMs >= startMs && positionMs <= endMs;
  }

  Map<String, dynamic> toJson() => {
        'id': id,
        'personTrackId': personTrackId,
        'startMs': startMs,
        'endMs': endMs,
        'style': style.name,
        'strength': strength,
        'keyframes':
            keyframes.map((item) => item.toJson()).toList(),
        'reviewed': reviewed,
        'reason': reason,
      };
}
```

---

12. Voice Protection Domain Code

voice_protection_track.dart

```
enum VoiceProtectionPreset {
  witnessMaximum,
  witnessNatural,
  childProtection,
  anonymousInterview,
  robot,
  deepVoice,
  cartoon,
  alien,
  mystery,
}

final class VoiceProtectionTrack {
  const VoiceProtectionTrack({
    required this.id,
    required this.speakerTrackId,
    required this.startMs,
    required this.endMs,
    required this.preset,
    required this.mode,
    required this.pitchShiftSemitones,
    required this.formantShift,
    required this.spectralWarp,
    required this.mix,
    required this.reviewed,
  });

  final String id;
  final String speakerTrackId;
  final int startMs;
  final int endMs;
  final VoiceProtectionPreset preset;
  final PrivacyProtectionMode mode;

  final double pitchShiftSemitones;
  final double formantShift;
  final double spectralWarp;

  /// 1 means only transformed voice should remain.
  final double mix;

  final bool reviewed;

  bool appliesAt(int positionMs) {
    return positionMs >= startMs && positionMs <= endMs;
  }

  Map<String, dynamic> toJson() => {
        'id': id,
        'speakerTrackId': speakerTrackId,
        'startMs': startMs,
        'endMs': endMs,
        'preset': preset.name,
        'mode': mode.name,
        'pitchShiftSemitones': pitchShiftSemitones,
        'formantShift': formantShift,
        'spectralWarp': spectralWarp,
        'mix': mix,
        'reviewed': reviewed,
      };
}
```

Required import:

```
import 'privacy_protection_mode.dart';
```

---

13. Person Detection Contract

person_detection_service.dart

```
final class DetectedPersonFrame {
  const DetectedPersonFrame({
    required this.timestampMs,
    required this.left,
    required this.top,
    required this.width,
    required this.height,
    required this.faceVisible,
    required this.confidence,
  });
```

```
  final int timestampMs;
  final double left;
  final double top;
  final double width;
  final double height;
  final bool faceVisible;
  final double confidence;
}

final class DetectedPersonTrack {
  const DetectedPersonTrack({
    required this.trackId,
    required this.frames,
    required this.minimumConfidence,
    required this.requiresReview,
  });

  final String trackId;
  final List<DetectedPersonFrame> frames;
  final double minimumConfidence;
  final bool requiresReview;
}

abstract interface class PersonDetectionService {
  Future<List<DetectedPersonTrack>> analyse({
    required String videoPath,
    required int samplingIntervalMs,
  });
}
```

On-device detection should be preferred for initial scanning where performance permits. Heavy re-identification and high-accuracy processing may use a governed server worker after creator approval.

---

14. Speaker Analysis Contract

speaker_analysis_service.dart

```
final class SpeakerSegment {
  const SpeakerSegment({
    required this.startMs,
    required this.endMs,
    required this.confidence,
  });

  final int startMs;
  final int endMs;
  final double confidence;
}

final class DetectedSpeakerTrack {
  const DetectedSpeakerTrack({
    required this.speakerId,
    required this.segments,
    required this.requiresReview,
  });

  final String speakerId;
  final List<SpeakerSegment> segments;
  final bool requiresReview;
}

abstract interface class SpeakerAnalysisService {
  Future<List<DetectedSpeakerTrack>> analyse({
    required String mediaPath,
  });
}
```

---

15. Privacy Project Model

privacy_shield_state.dart

```
import 'visual_protection_track.dart';
import 'voice_protection_track.dart';

final class PrivacyShieldState {
  const PrivacyShieldState({
    required this.visualTracks,
    required this.voiceTracks,
```

```
    required this.scanCompleted,
    required this.creatorReviewCompleted,
    required this.highRiskMode,
  });

  final List<VisualProtectionTrack> visualTracks;
  final List<VoiceProtectionTrack> voiceTracks;
  final bool scanCompleted;
  final bool creatorReviewCompleted;
  final bool highRiskMode;

  bool get allVisualTracksReviewed {
    return visualTracks.every((track) => track.reviewed);
  }

  bool get allVoiceTracksReviewed {
    return voiceTracks.every((track) => track.reviewed);
  }

  bool get readyForProtectedRender {
    return scanCompleted &&
        creatorReviewCompleted &&
        allVisualTracksReviewed &&
        allVoiceTracksReviewed;
  }

  Map<String, dynamic> toJson() => {
        'visualTracks':
            visualTracks.map((track) => track.toJson()).toList(),
        'voiceTracks':
            voiceTracks.map((track) => track.toJson()).toList(),
        'scanCompleted': scanCompleted,
        'creatorReviewCompleted': creatorReviewCompleted,
        'highRiskMode': highRiskMode,
      };
}
```

Add this state to `CreatorProject`.

---

16. Privacy Shield Controller

`privacy_shield_controller.dart`

```
import 'package:flutter/foundation.dart';
import 'package:uuid/uuid.dart';

import '../domain/privacy_shield_state.dart';
import '../domain/visual_protection_track.dart';
import '../domain/voice_protection_track.dart';
import '../services/person_detection_service.dart';
import '../services/speaker_analysis_service.dart';

final class PrivacyShieldController extends ChangeNotifier {
  PrivacyShieldController({
    required PersonDetectionService personDetection,
    required SpeakerAnalysisService speakerAnalysis,
    Uuid uuid = const Uuid(),
  })  : _personDetection = personDetection,
        _speakerAnalysis = speakerAnalysis,
        _uuid = uuid;

  final PersonDetectionService _personDetection;
  final SpeakerAnalysisService _speakerAnalysis;
  final Uuid _uuid;

  PrivacyShieldState _state = const PrivacyShieldState(
    visualTracks: [],
    voiceTracks: [],
    scanCompleted: false,
    creatorReviewCompleted: false,
    highRiskMode: false,
  );

  PrivacyShieldState get state => _state;

  bool _scanning = false;
  bool get scanning => _scanning;

  double _progress = 0;
  double get progress => _progress;

  Future<void> scan({
    required String videoPath,
```

```
  required bool includeVoice,
  required bool highRiskMode,
}) async {
  _scanning = true;
  _progress = 0;
  notifyListeners();

  try {
    final peopleFuture = _personDetection.analyse(
      videoPath: videoPath,
      samplingIntervalMs: highRiskMode ? 100 : 250,
    );

    final speakersFuture = includeVoice
        ? _speakerAnalysis.analyse(mediaPath: videoPath)
        : Future.value(<DetectedSpeakerTrack>[]);

    final people = await peopleFuture;
    _progress = 0.6;
    notifyListeners();

    final speakers = await speakersFuture;

    _state = PrivacyShieldState(
      visualTracks: people.map((person) {
        return VisualProtectionTrack(
          id: _uuid.v4(),
          personTrackId: person.trackId,
          startMs: person.frames.first.timestampMs,
          endMs: person.frames.last.timestampMs,
          style: highRiskMode
              ? VisualProtectionStyle.solidSilhouette
              : VisualProtectionStyle.gaussianBlur,
          strength: highRiskMode ? 1 : 0.75,
          keyframes: person.frames.map((frame) {
            return VisualProtectionKeyframe(
              timestampMs: frame.timestampMs,
              left: frame.left,
              top: frame.top,
              width: frame.width,
              height: frame.height,
              confidence: frame.confidence,
            );
          }).toList(growable: false),
          reviewed: false,
          reason: highRiskMode
              ? 'High-risk identity protection'
              : null,
        );
      }).toList(growable: false),
      voiceTracks: speakers.map((speaker) {
        return VoiceProtectionTrack(
          id: _uuid.v4(),
          speakerTrackId: speaker.speakerId,
          startMs: speaker.segments.first.startMs,
          endMs: speaker.segments.last.endMs,
          preset: VoiceProtectionPreset.anonymousInterview,
          mode: PrivacyProtectionMode.privacy,
          pitchShiftSemitones: -4,
          formantShift: 0.82,
          spectralWarp: 0.25,
          mix: 1,
          reviewed: false,
        );
      }).toList(growable: false),
      scanCompleted: true,
      creatorReviewCompleted: false,
      highRiskMode: highRiskMode,
    );

    _progress = 1;
  } finally {
    _scanning = false;
    notifyListeners();
  }
}

void setVisualProtection({
  required String trackId,
  required bool protected,
  VisualProtectionStyle? style,
  double? strength,
}) {
  final tracks = _state.visualTracks.map((track) {
    if (track.id != trackId) {
      return track;
    }
```

```
    return VisualProtectionTrack(
      id: track.id,
      personTrackId: track.personTrackId,
      startMs: track.startMs,
      endMs: track.endMs,
      style: style ?? track.style,
      strength: protected
          ? (strength ?? track.strength)
          : 0,
      keyframes: track.keyframes,
      reviewed: true,
      reason: track.reason,
    );
  }).toList(growable: false);

  _replaceState(visualTracks: tracks);
}

void setVoicePreset({
  required String trackId,
  required VoiceProtectionPreset preset,
}) {
  final tracks = _state.voiceTracks.map((track) {
    if (track.id != trackId) {
      return track;
    }

    final profile = _profileFor(preset);

    return VoiceProtectionTrack(
      id: track.id,
      speakerTrackId: track.speakerTrackId,
      startMs: track.startMs,
      endMs: track.endMs,
      preset: preset,
      mode: profile.mode,
      pitchShiftSemitones: profile.pitch,
      formantShift: profile.formant,
      spectralWarp: profile.spectralWarp,
      mix: profile.mix,
      reviewed: true,
    );
  }).toList(growable: false);

  _replaceState(voiceTracks: tracks);
}

void completeReview() {
  _state = PrivacyShieldState(
    visualTracks: _state.visualTracks,
    voiceTracks: _state.voiceTracks,
    scanCompleted: _state.scanCompleted,
    creatorReviewCompleted: true,
    highRiskMode: _state.highRiskMode,
  );

  notifyListeners();
}

void _replaceState({
  List<VisualProtectionTrack>? visualTracks,
  List<VoiceProtectionTrack>? voiceTracks,
}) {
  _state = PrivacyShieldState(
    visualTracks: visualTracks ?? _state.visualTracks,
    voiceTracks: voiceTracks ?? _state.voiceTracks,
    scanCompleted: _state.scanCompleted,
    creatorReviewCompleted:
        _state.creatorReviewCompleted,
    highRiskMode: _state.highRiskMode,
  );

  notifyListeners();
}

_VoiceProfile _profileFor(
  VoiceProtectionPreset preset,
) {
  return switch (preset) {
    VoiceProtectionPreset.witnessMaximum =>
      const _VoiceProfile(
        mode: PrivacyProtectionMode.privacy,
        pitch: -7,
        formant: 0.68,
        spectralWarp: 0.48,
        mix: 1,
```

```
        ),
      VoiceProtectionPreset.witnessNatural =>
        const _VoiceProfile(
          mode: PrivacyProtectionMode.privacy,
          pitch: -4,
          formant: 0.82,
          spectralWarp: 0.28,
          mix: 1,
        ),
      VoiceProtectionPreset.childProtection =>
        const _VoiceProfile(
          mode: PrivacyProtectionMode.privacy,
          pitch: -5,
          formant: 0.72,
          spectralWarp: 0.40,
          mix: 1,
        ),
      VoiceProtectionPreset.anonymousInterview =>
        const _VoiceProfile(
          mode: PrivacyProtectionMode.privacy,
          pitch: -3,
          formant: 0.86,
          spectralWarp: 0.22,
          mix: 1,
        ),
      VoiceProtectionPreset.robot =>
        const _VoiceProfile(
          mode: PrivacyProtectionMode.creative,
          pitch: -1,
          formant: 0.95,
          spectralWarp: 0.55,
          mix: 1,
        ),
      VoiceProtectionPreset.deepVoice =>
        const _VoiceProfile(
          mode: PrivacyProtectionMode.creative,
          pitch: -6,
          formant: 0.78,
          spectralWarp: 0.12,
          mix: 1,
        ),
      VoiceProtectionPreset.cartoon =>
        const _VoiceProfile(
          mode: PrivacyProtectionMode.creative,
          pitch: 6,
          formant: 1.24,
          spectralWarp: 0.18,
          mix: 1,
        ),
      VoiceProtectionPreset.alien =>
        const _VoiceProfile(
          mode: PrivacyProtectionMode.creative,
          pitch: 2,
          formant: 1.10,
          spectralWarp: 0.62,
          mix: 1,
        ),
      VoiceProtectionPreset.mystery =>
        const _VoiceProfile(
          mode: PrivacyProtectionMode.creative,
          pitch: -4,
          formant: 0.88,
          spectralWarp: 0.34,
          mix: 1,
        ),
    };
  }
}

final class _VoiceProfile {
  const _VoiceProfile({
    required this.mode,
    required this.pitch,
    required this.formant,
    required this.spectralWarp,
    required this.mix,
  });

  final PrivacyProtectionMode mode;
  final double pitch;
  final double formant;
  final double spectralWarp;
  final double mix;
}
```

---

17. Rendering Design

Visual blur rendering

For each person track:

1. Interpolate the bounding box between keyframes.
2. Expand the mask around the head or body.
3. Smooth movement to avoid shaking masks.
4. Apply blur/pixelation.
5. Render the effect permanently into the export.
6. Verify no unprotected frames remain.

Example FFmpeg design for a fixed region:

```
ffmpeg -i input.mp4 \
-filter_complex \
"[0:v]split=2[base][face];
 [face]crop=300:300:100:100,boxblur=30:15[blurred];
 [base][blurred]overlay=100:100:enable='between(t,1.0,8.5)'" \
-c:a copy output.mp4
```

Moving people require dynamically generated masks, per-frame coordinates, or a pre-rendered alpha-mask video. The recommended production approach is:

```
Detection worker
      ↓
Mask video with alpha channel
      ↓
FFmpeg compositing
      ↓
Protected export
```

Voice processing

Pitch-only FFmpeg processing is acceptable only for Creative Mode prototypes.

Privacy Mode should use a dedicated voice-anonymization processor capable of formant and spectral transformation.

The final mix must not retain the original speaker below the transformed layer when `mix = 1`.

---

18. Protected Render Contract

`privacy_render_service.dart`

```
import '../domain/privacy_shield_state.dart';

abstract interface class PrivacyRenderService {
  Stream<double> get progress;

  Future<String> renderProtectedVideo({
    required String inputPath,
    required String outputPath,
    required PrivacyShieldState privacy,
  });

  Future<void> cancel();
}
```

Server rendering request

```
export interface PrivacyRenderRequest {
  projectId: string;
  sourceAssetId: string;
  highRiskMode: boolean;

  visualTracks: Array<{
    id: string;
    personTrackId: string;
    style:
      | "gaussianBlur"
      | "pixelate"
      | "solidSilhouette"
      | "mosaic"
      | "emojiMask"
      | "avatarMask";
    strength: number;
```

```
    startMs: number;
    endMs: number;
    keyframes: Array<{
      timestampMs: number;
      left: number;
      top: number;
      width: number;
      height: number;
      confidence: number;
    }>;
  }>;

  voiceTracks: Array<{
    id: string;
    speakerTrackId: string;
    preset: string;
    mode: "privacy" | "creative";
    startMs: number;
    endMs: number;
    pitchShiftSemitones: number;
    formantShift: number;
    spectralWarp: number;
    mix: number;
  }>;
}
```

---

19. Privacy Publication Gate

Flutter gate extension

```
final class PrivacyPublicationEvidence {
  const PrivacyPublicationEvidence({
    required this.privacyShieldRequired,
    required this.scanCompleted,
    required this.reviewCompleted,
    required this.protectedRenderCompleted,
    required this.lowConfidenceSectionsResolved,
    required this.originalMediaExcludedFromPublication,
  });

  final bool privacyShieldRequired;
  final bool scanCompleted;
  final bool reviewCompleted;
  final bool protectedRenderCompleted;
  final bool lowConfidenceSectionsResolved;
  final bool originalMediaExcludedFromPublication;
}
```

Add to the publication gate:

```
void evaluatePrivacy(
  PrivacyPublicationEvidence privacy,
  List<String> blockers,
) {
  if (!privacy.privacyShieldRequired) {
    return;
  }

  if (!privacy.scanCompleted) {
    blockers.add('Privacy Shield scanning is incomplete.');
  }

  if (!privacy.reviewCompleted) {
    blockers.add(
      'Every detected person and speaker requires review.',
    );
  }

  if (!privacy.protectedRenderCompleted) {
    blockers.add('Protected rendering is incomplete.');
  }

  if (!privacy.lowConfidenceSectionsResolved) {
    blockers.add(
      'Low-confidence privacy sections require manual review.',
    );
  }

  if (!privacy.originalMediaExcludedFromPublication) {
    blockers.add(
      'Original unprotected media remains linked to publication.',
    );
  }
```

```
}
```

---

20. Server-Side Privacy Gate

```
export interface PrivacyGateInput {
  required: boolean;
  scanCompleted: boolean;
  reviewCompleted: boolean;
  protectedRenderCompleted: boolean;
  lowConfidenceResolved: boolean;
  originalAssetPublicationBlocked: boolean;
  outputAssetId?: string;
}

export function evaluatePrivacyGate(
  input: PrivacyGateInput,
): string[] {
  if (!input.required) {
    return [];
  }

  const blockers: string[] = [];

  if (!input.scanCompleted) {
    blockers.push("Privacy scan is incomplete.");
  }

  if (!input.reviewCompleted) {
    blockers.push(
      "Detected people and speakers require creator review.",
    );
  }

  if (!input.protectedRenderCompleted) {
    blockers.push(
      "Protected video rendering is incomplete.",
    );
  }

  if (!input.lowConfidenceResolved) {
    blockers.push(
      "Low-confidence protection segments remain unresolved.",
    );
  }

  if (!input.originalAssetPublicationBlocked) {
    blockers.push(
      "The original unprotected asset is still publishable.",
    );
  }

  if (!input.outputAssetId) {
    blockers.push(
      "Protected output asset is missing.",
    );
  }

  return blockers;
}
```

The server must publish only the protected output asset, never the unprotected original.

---

21. Privacy Shield UI

The user should see:

Detection screen

- Number of people detected
- Number of speakers detected
- Timeline tracks
- Confidence warnings
- “Protect everyone” action
- “Protect selected people” action
- “Protect all voices” action

Person review card

```
Person 1
Visible: 00:02–00:18
Confidence: 96%

[Protect] [Consent confirmed] [Review frames]
```

Speaker review card

```
Speaker 1
Active: 00:04–00:26

[Protected Witness]
[Anonymous Interview]
[Creative Voice]
[No Protection]
```

Final warning

Review the complete protected preview before publication. Automated detection may miss people, reflections, identifying objects, or portions of speech.

---

22. Security and Storage Rules

For high-risk projects:

- Original media should be stored in a private restricted location.
- Protected output should use a new asset ID.
- Public videos must reference only the protected asset.
- Original media should never inherit public-read permissions.
- Signed URL duration should be minimized.
- Access must be audited.
- Download and export permissions should be role-controlled.
- Optional automatic deletion of originals should be available after successful protected rendering.
- Encryption keys and processing logs should be separated from public metadata.

Suggested paths:

```
private-creator-source/{ownerId}/{projectId}/original.mp4
private-privacy-masks/{ownerId}/{projectId}/mask.mov
protected-creator-output/{ownerId}/{projectId}/protected.mp4
```

---

23. Analytics

Required events:

```
privacy_scan_started
privacy_scan_completed
privacy_person_detected
privacy_speaker_detected
privacy_visual_track_enabled
privacy_voice_track_enabled
privacy_low_confidence_found
privacy_manual_correction
privacy_protected_render_started
privacy_protected_render_completed
privacy_protected_render_failed
privacy_publication_blocked
privacy_publication_approved
privacy_original_deleted
```

Do not send face embeddings, voiceprints, original audio, or sensitive identity details into general-purpose analytics.

---

24. Required Validation

Visual protection

- One stationary face
- One moving face
- Multiple people
- Side profile
- Face temporarily hidden
- Person exits and returns

- Reflection
- Low light
- Rapid movement
- Partial body
- Imported video
- Recorded video

Voice protection

- One speaker
- Multiple speakers
- Overlapping speakers
- Loud background noise
- Whispering
- Singing
- Different accents
- Phone-quality audio
- Protected segment followed by normal segment
- Creative voice effect
- Witness Maximum mode

Privacy leak review

- Frame-by-frame face exposure
- Original voice leakage
- Reflections
- Identifying text
- Metadata
- Original-source accessibility
- Download permissions
- Publication reference points only to protected output

---

25. Release Strategy

Phase 1 — Controlled Visual Shield

Enable:

- Face/person detection
- Select person
- Strong blur
- Pixelation
- Protect everyone
- Manual correction
- Protected rendering

Phase 2 — Controlled Voice Shield

Enable:

- Manual timeline voice protection
- Single-speaker anonymization
- Witness Maximum
- Anonymous Interview
- Creative presets

Phase 3 — Multi-Person Intelligence

Enable:

- Stable cross-scene tracking
- Multiple selected people
- Speaker diarization
- Speaker-specific transformation
- Possible-minor review prompts

Phase 4 — High-Risk Certification

Enable only after an independent security/privacy review:

- Protected Witness mode

- Automatic metadata stripping
- Original-source lockdown
- High-risk account workflow
- Audit and evidence package
- Restricted-role access

Recommended Launch Decision

Both features should be introduced, but not presented as ordinary decorative effects.

They should launch as a dedicated safety capability:

YohPal Privacy Shield

Recommended public positioning:

Protect identities before publishing. Blur selected people, obscure voices, and review privacy risks directly inside Creator Studio.

For protected witnesses or other high-risk use, the application must clearly state:

Identity protection reduces recognition risk but cannot guarantee absolute anonymity. Review the complete video and remove contextual details that may reveal identity.

Next Smart Move

Build YOHPAL LIVE CREATOR STUDIO RELEASE 1.1A-RC2 — PRIVACY SHIELD PILOT, adding person detection and tracking, selectable face/person blurring, manual mask correction, speaker timeline detection, Witness Maximum and Creative voice presets, protected rendering, original-source isolation, and server-enforced privacy publication gates. Keep the capability allow-listed until Android and iPhone testing proves that no protected person appears unblurred, no original protected voice leaks into the final mix, and only the protected output can be published.